2025 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-0647-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/HPCA61900.2025.00100

ARTEMIS: Agile Discovery of Efficient Real-Time Systems-on-Chips in the Heterogeneous Era

Subhankar Pal*†, Aporva Amarnath*†, Behzad Boroujerdian‡, Augusto Vega†, Alper Buyuktosunoglu†, John-David Wellman†, Vijay Janapa Reddi§, and Pradip Bose†

IBM T. J. Watson Research Center, NY, USA† University of Texas, Austin, TX, USA‡ Harvard University, MA, USA§
Email: Subhankar.Pal@ibm.com
*Equal contributors

***Abstract*—Heterogeneous systems-on-chips (SoCs) are pivotal for real-time applications like autonomous driving, as they blend the versatility of CPUs with the efficiency of accelerator IPs. However, evolving application demands necessitate domain-specific SoCs to meet real-time deadlines within strict power and area constraints. While prior research focused on microarchitectural optimizations, overlooking broader system-level considerations can lead to suboptimal design decisions. Thus, there is a need to elevate the abstraction level of design space exploration (DSE) to the SoC level. However, SoC-level DSE is challenging due to the vast design space, encompassing microarchitectural parameters and dynamic task-to-hardware mapping choices based on run-time characteristics and real-time constraints.**

This paper proposes a systematic and agile methodology, called ARTEMIS, for efficient DSE of real-time, domain-specific SoCs that are constrained by task deadlines, power, and area. The core concept involves integrating a dynamic SoC scheduler to reduce the design space by eliminating the mapping dimension. Enhanced scheduling policies, incorporating techniques like task procrastination and memory-traffic/energy awareness, expedite navigation through the pruned design space. Additionally, DSE heuristics are optimized with real-time deadline and power/area-aware ranking mechanisms. ARTEMIS is evaluated on autonomous vehicle (AV) and augmented/virtual reality (AR/VR) applications, and additionally validated on an FPGA. Compared to the state-of-the-art, DSE using ARTEMIS converges 5.1–12.8× faster, while yielding SoCs that meet 100% real-time deadlines with 1.2–3× better throughput at iso-area or up to 2.4× lower area for at iso-input-rate. ARTEMIS thus enables DSE of large designs with tractable simulation resources, without compromising on the power-performance-area metrics of the explored SoC design.

I. Introduction

Real-time applications, *e.g.,* in autonomous vehicles (AVs) and augmented/virtual reality (AR/VR), have gained tremendous traction. These applications, subject to strict deadlines and power and area constraints, rely heavily on heterogeneous system-on-chips (SoCs), which combine CPUs and/or GPUs, and hardware accelerator IPs. However, the design space of such highly-heterogeneous SoCs is vast due to a multitude of variables, *e.g.,* hardware composition, topology, clock/voltage domains, *etc.* Further, the choices of mapping the application's tasks onto the SoC adds yet another vast dimension of parameters for design space exploration (DSE).

Several machine learning (ML) based search algorithms have been proposed in prior work to navigate large design spaces (*e.g.,* [8], [12], [33], [38]). Interestingly, a recent work, ArchGym [30], concludes that the efficacy of popular ML-based search algorithms tend to converge given enough samples. Therefore, the size of the design space poses a greater bottleneck than the choice of the search algorithm. Prior works have attempted to minimize the design space by filtering out invalid design points, gathering metrics from a system simulator to guide the exploration of the next design point [8], and through the use of static mapping policies [27]. Yet, these methods do not significantly reduce the design space, reduce it to a sub-optimal space, and/or fail to converge or produce design(s) that meets all real-time constraints.

In our work, we leverage two key insights overlooked in previous studies.

- *First*, prior works consider task to processing element (PE) mapping choices as a static DSE parameter, thus compounding the design space of with heterogeneous SoCs, especially for real-time systems. Eliminating this dimension would result in several orders-of-magnitude fewer design points; $\sim 10^{20}$ fewer for a practical design, based on our estimates.
- *Second*, prior works collect summarized statistics at the end of each design point's simulation and attempt to identify bottlenecks post-simulation. This approach misses out on opportunities to leverage accurate dynamic runtime insights obtained during the simulated execution to guide the DSE, or better, to manipulate the runtime to generate DSE hints.

Motivated by these insights, we introduce **ARTEMIS**: A Real Time SoC Explorer with Mapping Inferred by the Scheduler. It is a framework for fast early-stage DSE of SoCs under latency, power and area constraints, and is particularly useful for domain-specific SoC design to accelerate real-time applications.

The key features proposed in this work are:

- *Elimination of the task-to-PE mapping dimension.* ARTEMIS introduces the use of a dynamic SoC scheduler[1], that is synergistic with the post-fabrication SoC scheduler/governor, to perform mapping during the simulation of each design point in the iterative DSE

[1]An SoC scheduler governs the operations of *when* and *where* tasks execute and *how* the data moves. A dynamic SoC scheduler determines these operations at run-time.

979-8-3315-0647-6/25/$31.00 ©2025 IEEE
DOI 10.1109/HPCA61900.2025.00100

TABLE I
COMPARISON WITH PRIOR WORK.

Reference	Real-Time Application Support	Simulation Metrics for Design Point Selection	Real-Time Heuristics for Design Point Selection	Use of Scheduler for Mapping	Use of Scheduler for Design Point Selection
[26], [33], [37]	×	×	×	×	×
[16], [28]	×	×	×	×	×
[12]	×	✓	×	×	×
[13]	✓	×	×	×	×
[6], [39]	×	✓	×	Static	×
[29]	✓	✓	×	Static	×
[8]	✓	✓	×	×	×
[27], [30]	✓	✓	Limited	Static	×
ARTEMIS	✓	✓	✓	Dynamic	✓

process. This leads to a pruned design space that is several orders-of-magnitude smaller than if the mapping is explored as a separate dimension during the DSE itself.

- *Leveraging dynamic runtime scheduling for DSE.* In order to expose bottleneck task(s) to the explorer, we propose an real-time heterogeneity-aware scheduling policy called ART_{DSE} that introduces *task procrastination*, in conjunction with energy-aware and memory-traffic–aware scheduling optimizations, to deter the exploration of architecturally over-provisioned design points.
- *Leveraging real-time statistics for bottleneck identification.* ARTEMIS uses runtime information, *e.g.,* task latencies, task deadlines and waiting times, to create ranking methods for selecting the metric, task, and hardware block to optimize, and the block to optimize with.

Overall, our experiments demonstrate that ARTEMIS outputs SoC designs that meet 100% deadlines and sustain up to **3**× higher throughput for the same area (or **2.4**× lower area for the same throughput) for an AV application, over the state-of-the-art (SoTA) [8], [27]. ARTEMIS converges in its DSE loop **8.4–12.8**× and **5.1**× faster, for the AV and AR/VR workloads *resp.*, while producing more efficient SoCs that meet **100%** of real-time deadlines.

While we explore the benefits of ARTEMIS for these two domains in this work, the framework is sufficiently generic to cater to any real-time domain. ARTEMIS is publicly available at https://github.com/IBM/ARTEMIS-DSE and we posit it to be used as an integral part of an architect's workbench to automate future real-time system design.

II. BACKGROUND AND RELATED WORK

We first motivate the challenges associated with DSE for heterogeneous SoCs in the context of real-time applications, and discuss prior work in this area. We then provide background on the characteristics of real-time applications, using the example of AVs. We conclude with discussion of prior work on mapping and scheduling for heterogeneous SoCs.

A. Challenges with SoC-Level DSE

The design space of heterogeneous SoCs for applications is extensive, owing to *(i)* hardware knobs, such as heterogeneity level, sizes of memory and interconnects, clock speed and *(ii)* application-specific constraints that are determined by operational factors, such as AV speed and battery capacity. Given this level of complexity, it is imperative to make careful design choices that *holistically* meet application requirements and PPA constraints, avoiding late-stage changes with significant re-development effort.

Early-stage DSE of SoCs has two aspects. The first is **architectural exploration** to determine the number of processing elements (PEs) and memories, types of PEs (*e.g.,* CPUs or IPs) and memories (*e.g.,* SRAM or DRAM), the network on-chip (NoC) topology, clock speeds, and bus widths, among others.

The second aspect includes the task-to-PE assignments, traffic routing, memory (buffer) allocation, *etc.*, collectively known as **mapping**. Given the vast number of parameters and their possibilities, the main challenge with DSE for heterogeneous systems lies with *scalability*. In particular, DSE with growing number of tasks leads to an exponential increase in the size of the DSE state space, due to an explosion in the mapping possibilities. Figure 1 (blue grid) provides an estimation of the vast design space size, which, even for a modest number of PEs and tasks, can be as large as the estimated number of stars in the universe! This makes it challenging for any exploration mechanism to converge within tractable time. Consider this in the context of ERA (an autonomous vehicle workload, described in Sec. IV-A) for 3 DAGs and a 3×3 mesh-based SoC. The number of mapping choices[2] here is 10^{20}, which would take ∼3,200 years to explore at a rate of 1B choices/second! Therefore, eliminating this is important. More generally, as seen in Figure 1, the design space size can be vastly reduced, by $10^{20}\times$ or more, if the task-to-PE mapping dimension can be eliminated.

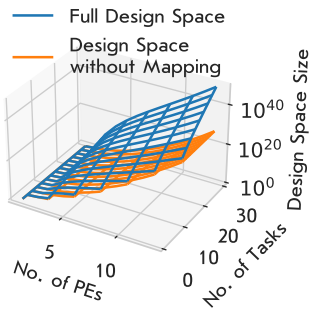

Fig. 1. Design space size for exploration of heterogeneous SoCs, assuming 4 generic hardware knobs per PE (*e.g.,* clock speed). Our technique eliminates the mapping dimension by delegating task-to-PE-mapping to the SoC scheduler.

B. DSE frameworks for SoC Design

Several frameworks have been proposed for the DSE of heterogeneous SoCs. Tab. I shows a qualitative comparison of

[2]The number of task-to-PE mapping choices is computed as $S(n_{tasks} \cdot n_{DAGs}, n_{PEs}) \cdot (n_{PEs}!)$, where S is the Stirling partition number.

some of these with ARTEMIS, in terms of their applicability to real-time domains, features such as the use of simulation metrics and real-time heuristics for optimizing DSE, and the use of schedulers for mapping to reduce the design space and to identify task bottlenecks during execution.

1) Search Algorithm Optimizations.: These frameworks can further be categorized into two, based on the optimization method used. *Multi-objective linear solver*-based solutions, such as [28], [29], [37] focus on designing optimization functions to derive a Pareto-optimal solution.

The second category uses *multi-objective evolutionary algorithms*, such as simulated annealing (SA) [8], TABU search [16], genetic algorithms (GA) [13], [33], Pareto Hyper-Volume (PHV) guided search [12], reinforcement learning (RL) [30] and hybrid approaches [26]. Search algorithms are terminated using heuristics that either derive energy-efficient solutions [12], [16], [33] or attempt to meet real-time constraints [8], [13], [30]. Prior work [8] further propose the use architectural insights from a system simulator for intelligent selection of nearest-neighbor design points during DSE. However, the vastness of the SoC design space for real-time domains (Sec. II-A) leads to long convergence times, even with such mechanisms. In contrast, ARTEMIS drastically reduces the size of the design space itself by delegating task-to-PE mapping to an real-time scheduler. Furthermore, it adapts the scheduler to aid in DSE by identifying tasks that require optimizations, thereby accelerating the search process.

2) DSE using Static Mapping Techniques: Prior works have also explored the use of static mapping algorithms for the design of SoCs and coarse-grained reconfigurable arrays (CGRAs). [27] uses a two-step approach of pruning the design space using real-time constraints and simulating the candidate designs to arrive at the best solution. [29] proposes the use of a static scheduling policy to reduce the design space to be explored for multimedia applications. [6], [39] propose the use of static algorithms to map nodes of a dataflow graph (DFG) onto the PEs of CGRAs. However, these static methods perform DSE for a single DFG at a time and do not consider dynamic traces consisting of multiple DFGs that are more representative of real-time applications.

Moreover, static mapping algorithms have limitations in terms of sustaining high throughput while meeting deadlines, as such policies do not account for dynamic run-time information, *e.g.,* task wait times and task deadlines, of the workloads. In contrast, ARTEMIS proposes the use of a dynamic scheduler that accounts for such information to perform task prioritization and mapping. While it is possible to extract run-time information from a static scheduler, the inability to modify the schedule on-the-fly may reduce the relevance of such information for the static case. Prior works using a static approach fix a schedule for each task, run the simulation, and then use post-simulation metrics to determine the bottleneck task, and the block and metric to improve. Because of a predetermined schedule, the actual bottleneck task could lead to a cascading effect leading to other tasks missing their deadlines, which may go away if the bottleneck task were to be accelerated. In our work, using a technique called task procrastination, *e.g.,* we (1) isolate any tasks that will not meet their deadlines based on the availability of resources, and (2) execute these tasks at a later time (procrastinate) when they will not perturb the execution of any other tasks that *could* meet their deadlines. This dynamism allows us to capture a more accurate representation of what the system state would be *if* the bottleneck task were to be accelerated, and thereby generate a representative set of dynamic run-time information.

C. Primer on Real-Time Applications

Real-time applications are pervasive in the modern world, in domains ranging from AVs (drones, cars, *etc.*) to real-time signal processing, AR/VR, and so on. Different applications have different constraints. AVs, for example, are required to abide by strict safety regulations to ensure the safety of human life and property [24]. According to ISO 26262, tasks associated with automotive safety integrity level (ASIL) B, C, and D are associated with safety hazards, requiring 100% of deadlines to be met to ensure system safety. This aligns with the definition of hard real-time tasks. In AV systems, tasks related to braking, obstacle detection, and decision-making cannot miss deadlines, as this could lead to safety-critical failures. While ASIL-A tasks can have firm or soft deadlines, repeated deadline misses in such cases could lead to promotion to a higher criticality level to ensure system robustness and reliability.

Other applications, such as signal processing applications and certain use-cases in AR/VR, may have soft deadlines, and it may be desired to have a balanced design point that delivers good user experience under low power consumption.

Real-time applications consist of tasks that are executed based on a control-flow graph (CFG) of the application, which can be extracted by a compiler. The CFG is further divided into directed-acyclic graphs (DAGs), where a *node* represents a task and an *edge* represents the data dependencies and data movement between tasks. The application is then modeled as DAGs "arriving" at the SoC. While the CFG itself may be static, the arrival times and interleaving of tasks across the derived DAGs are dynamic, as they are resolved at run-time.

Consider an example of a connected AV traveling on the road, whose workload's DAG representation is shown in Figure 6 (see: ERA). It uses radar to detect the nearest obstacle in a given lane, computer vision (CV) to identify the type of obstacle, and Viterbi decoding to decode messages from other AVs, *e.g.,* via Wi-Fi. Real-time domains necessitate the use of heterogeneous SoCs and dynamic constraint-aware scheduling techniques to meet stringent requirements for latency and power at the edge. As futuristic applications get even more complex, it is imperative to have a methodology for fast exploration of the large design space of heterogeneous SoCs.

D. Heterogeneous Real-Time SoC Scheduling

Heterogeneous SoCs for real-time applications are typically deployed with a dynamic SoC scheduler. A dynamic scheduler is essential for real-time applications due to its ability to

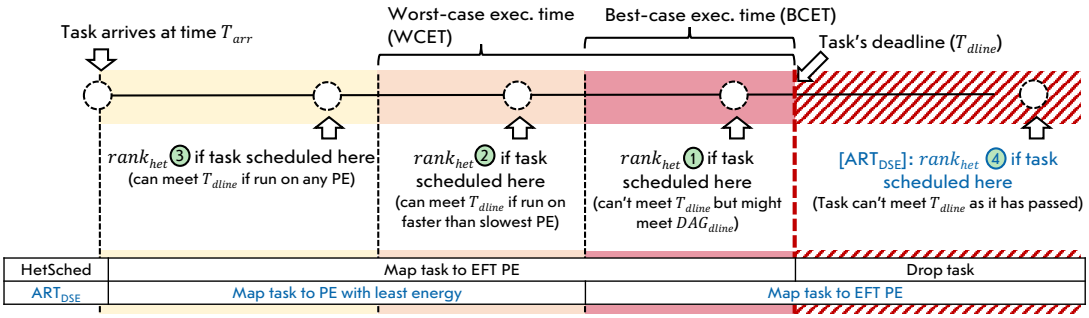

Fig. 2. Task prioritization (top) and mapping (bottom table) using available slack in HetSched and ART_{DSE} [this work]. The timeline from task arrival to deadline and beyond is divided into four zones: yellow, orange, red and hatched-red.

adapt to varying workloads and task arrival patterns, ensuring efficient task scheduling based on current system conditions. It plays a critical role in meeting strict deadlines by prioritizing tasks according to their urgency, thus ensuring timely execution. Additionally, dynamic scheduling optimizes system resources like CPU, memory, and I/O bandwidth by allocating them based on task needs and system load, thereby enhancing overall performance and resource utilization. Dynamic SoC schedulers may be implemented in software [9], [14], [19] or hardware [11], [31].

Scheduling policies for real-time applications (represented as multiple parallel/overlapping DAGs) typically prioritize tasks using earliest arrival time first, earliest-deadline first (EDF) or least-laxity first [4], [42], [43]. Specifically, we consider a SoTA real-time scheduler built for heterogeneous SoCs, called HetSched [4]. It performs *task prioritization* and *task-to-PE mapping* based on dynamic real-time knowledge, *e.g.,* slack[3], variable execution times across PE types, *etc.*

Task Prioritization. HetSched uses a two-level prioritization scheme through a discrete $rank_{het}$ and a continuous $rank_{hom}$. Fig. 2 shows four zones that a task could lie in when it is scheduled with respect to its task deadline. Note that each task in the DAG derives its task deadline as a fraction of the DAG's slack. This fraction is defined as $\min_{\text{all paths}} \frac{BCET_j}{\sum_{\text{all t}_i \text{ after t}_j \text{ in path:t}_j \in \text{path}} BCET_i}$. $BCET$ (best-case execution time) and $WCET$ (worse-case execution time) represent the execution time of the task on the fastest PE and slowest PE in the SoC, *resp.* HetSched assigns the lowest priority to the task in the yellow zone with $rank_{het}$=3. As the task moves into zones closer to the deadline, it is assigned a higher priority through decreasing $rank_{het}$ (yellow→orange→red zone). Note that HetSched does not rank or execute tasks that have missed their deadlines (hatched zone). Tasks within each zone are then prioritized based on their $rank_{hom}$, which is determined by the amount of slack available until the task's deadline if executed on a specific type of PE.

Task-to-PE Mapping. Ordered tasks are mapped using the earliest finish time (EFT) scheme (see table in Fig. 2). The scheduler estimates the execution time of the task on all eligible PEs in the system, considering idle/busy states and waiting tasks. The task is then assigned to the PE that can complete it in the shortest time.

Although HetSched sustains a higher throughput over prior policies for real-time applications while meeting all deadlines, it is not equipped to explore the design space to improve upon tasks that are bottlenecked due to unavailability of suitable PEs (in terms of power/performance) or due to a memory/NoC bottlenecks. ARTEMIS builds upon these opportunities, as we describe in the next section.

[3]A task's (or DAG's) slack is the time remaining until the task's (or DAG's) deadline.

III. The ARTEMIS Framework

A. Overview

At a high level, ARTEMIS consists of an exploration and a simulation framework that interact at various stages (Fig. 3). A DAG-level description of the workload, its tasks, and data dependencies, along with the constraints of the application and the hardware are sent as inputs to the framework. Additionally, a set of pre-characterized components, such as PEs, interconnects (ICs) and different memory types, are specified to the framework and are used to construct points in the design space to be explored.

ARTEMIS begins with a seed SoC design point, *e.g.,* one CPU/IC/memory, and incrementally transforms it to an optimized design point that meets the pre-specified constraints. It consists of a simulator that takes as input the design point and workload descriptions, and simulates the execution of the workload on the design point. A key component of the simulator is a scheduler and a set of scheduling policies that are responsible for both task prioritization (when to execute) and task mapping (where to execute). As stated earlier, our approach deliberately excludes the mapping from the design space and aids the DSE process (Sec. III-B).

During simulation, several statistics, *e.g.,* task waiting times, task deadlines, *etc.*, are computed and fed into the ARTEMIS explorer. The explorer uses real-time constraints aware policies to first identify the violated metric, among latency, power, and area (Sec. III-C1). Then it calculates which task needs to execute faster (or more efficiently) to improve this metric

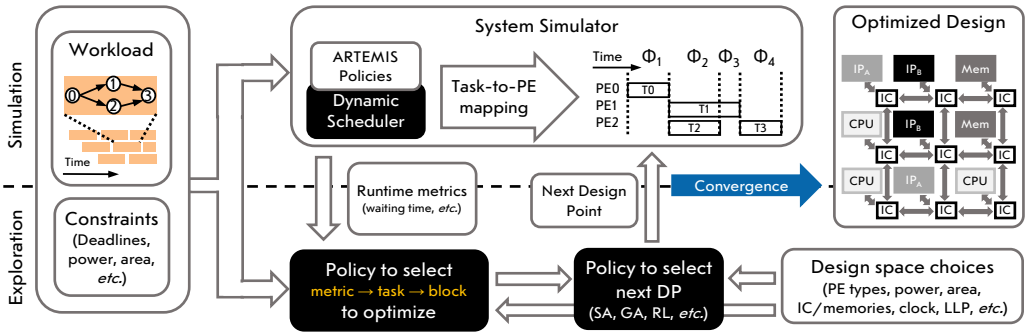

Fig. 3. Overview of our ARTEMIS framework. It inputs a library of pre-characterized design space choices for hardware components, SoC power-performance-area constraints, and an application model, to produce an optimized SoC design. IC=interconnect, IP=intellectual property (accelerator).

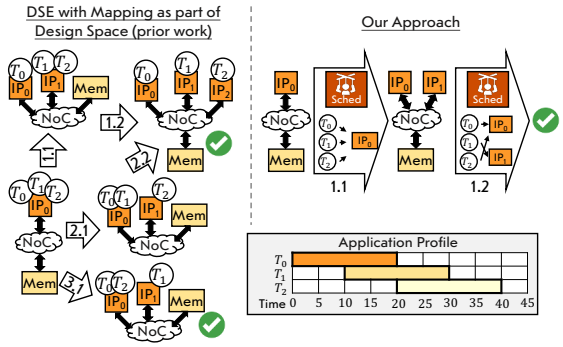

Fig. 4. DSE of hardware and mapping versus our approach that uses a real-time scheduler for mapping. Our method reduces the design space size to $^2/_5{}^{\text{th}}$ in this toy example.

(Sec. III-C2). It then examines the blocks that the scheduler mapped the task to and selects a block to improve (Sec. III-C3). The explorer then uses an exploration algorithm to generate the next design point and simulate it. This process continues until ARTEMIS encounters a design point that meets deadlines for all DAGs, and whose power and area are within the design constraints. Our methodology (Sec. IV-D) ensures that a design point that meets all DAGs' deadlines during DSE (for a specific DAG arrival rate) would also meet all deadlines on the field, *i.e.,* when it encounters a larger workload.

B. Adapting Scheduling Policies for DSE

One of our key insights is that a dynamic scheduling policy for the DSE of real-time SoCs would dramatically reduce the design space size and help navigate the pruned space effectively (Sec. I). In this section, we first highlight the necessity of using a real-time scheduler in the DSE process, emphasizing how this approach helps in faster convergence. We then delve into our primary contributions, showcasing how we have tailored a dynamic scheduler for DSE to discover an SoC design that meets real-time constraints without over-provisioning the design point.

1) Motivation: Consider the toy example illustrated in Figure 4, where the application consists of three homogeneous tasks (same deadline – 20 timesteps in this example) arriving 10 timesteps apart. The approach of DSE with mapping is shown at the left. The first move adds a new PE into the system (termed as "fork"), with $\binom{3}{2} \cdot 1 = 3$ possibilities for the task-to-PE mapping (1.1, 2.1, and 3.1). The outputs of 1.1 and 2.1 do not meet the deadlines for any of $\{T_0, T_1, T_2\}$, because one of the PEs serializes two concurrently-available tasks. The output of 3.1 meets all deadlines because T_2 arrives right after T_0 and they do not overlap when mapped to the same PE. If the explorer goes along paths 1.1 or 2.1, one of the PEs is forked yet again. The output design point of either of the steps (1.2 and 2.2) meets all tasks' deadlines. This solution, however, is over-provisioned in terms of SoC power and area, because it has three PEs instead of two (the optimal for this setup). Thus, assuming a uniform probability of selecting between steps 1.1-3.1, the explorer converges to the optimal design with a probability of only 33%.

Our Approach. We directly allow the SoC scheduler, during simulation, to decide the task-to-PE mapping, instead of exploring it statically. In Figure 4 (right), at step 1.1, the scheduler first schedules T_0 to PE_0. When T_1 arrives at time $t = 10$, it calculates and finds that T_1 will miss its deadline ($t = 30$) if scheduled onto PE_0, and thus schedules it onto PE_1. Finally, upon arrival of task T_2, the scheduler realizes that T_0 has just completed, and it is free to execute T_2 *in time for* T_2 *to meet its deadline*. The DSE converges at this design point, and with a minimal SoC design. This example illustrates that a good scheduling policy can not only reduce the exploration time, but also improve the quality of the output SoC.

2) Key Contributions: We now describe our proposed scheduling policy, ART_{DSE}, that integrates several features to improve the DSE process. This includes the notion of *task procrastination* to expose tasks that need optimization in the next design point. We also incorporate *low-cost NoC traffic awareness* and *PE-energy awareness* to enhance our scheduling decisions, thereby improving the DSE convergence.

Task Procrastination. To make ART_{DSE} useful in aiding in the DSE process, we choose to execute tasks that have failed to meet their deadlines, rather than drop them (see hatched region and table in Fig. 2). We assign them the lowest priority of

$rank_{het}$=4, effectively procrastinating their execution relative to other tasks. This strategy ensures best-effort execution of remaining tasks that are likely to meet their deadlines. We then utilize $rank_{hom}$ to de-prioritize/procrastinate tasks with larger deadline overruns over those with smaller ones. This approach enables our DSE task selection logic (Sec. III-C2) to prioritize procrastinated tasks with $rank_{het}$=4 and longer wait times for improvement in the next design point.

In order to ensure that the bottleneck tasks accurately get assigned a $rank_{het}$ of 4, we implement the following enhancements to HetSched and encapsulate these in our ART_{DSE} policy.

Energy-Aware Scheduling. With reference to Fig. 2, if a task lies in the orange or yellow zones when it is scheduled, it can potentially run on a low-power but slower PE and still meet its deadline *i.e.,* the slack can be converted into power savings. With this insight, ART_{DSE} maps tasks that have a $rank_{het}$ of 2 or 3 onto the PE with the least energy. Further, we exploit frequency-level heterogeneity by mapping tasks that have a $rank_{het}$ of 2 to slower-clocked IPs.

NoC Traffic-Aware Scheduling. We incorporate memory-traffic aware scheduling by estimating the expected NoC traffic before scheduling. For this, we track for each memory block the tasks that are reading from/writing to the block, and their data movements. This is used to estimate the *effective* bandwidth available for scheduling a new task on a potential PE, which is then used to alter the ranks in Fig. 2 as the $BCET$ and $WCET$ now vary depending on the state of the system. This mechanism can, thus, bump the $rank_{het}$ of a memory-bound task from 1 to 4, and cause the explorer to prioritize optimizing this task through task procrastination.

C. Real-Time Aware Selection Policies for DSE

1) Selection of Metric to Optimize: The metric-for-optimization in the next DSE iteration is selected as the one that has the largest distance among the following functions:

$$dist_{lat} = w_{lat} \cdot dist'_{lat}; \quad dist'_{lat} = \sum_{\text{all DAGs}} \frac{-slack_{DAG_k}}{T_{dline,DAG_k}} \quad (1)$$

$$dist_{pow} = w_{pow} \cdot [\frac{1}{pow_{budget}} \cdot (\max_{\text{all DAGs}} pow_{max,DAG_k} - pow_{budget})] \quad (2)$$

$$dist_{area} = w_{area} \cdot [\frac{1}{area_{budget}} \cdot (\sum_{\text{all blocks}} area_b - area_{budget})] \quad (3)$$

Here, the slack for executing DAG k, $slack_{DAG_k}$ is given by $T_{dline,DAG_k} - (T_{compl,DAG_k} - T_{arr,DAG_k})$, where T_{dline,DAG_k}, T_{compl,DAG_k}, and T_{arr,DAG_k} are its deadline, completion time, and arrival time, respectively, and pow_{max,DAG_k} is the peak SoC power during its execution. $area_b$ is the area of block b, while pow_{budget} and $area_{budget}$ are the power and area budget constraints, respectively. We set a large initial value for w_{lat} ($\gg w_{pow}, w_{area}$) and drop it exponentially as the DSE proceeds; this encourages a final design point that meets maximal DAG deadlines.

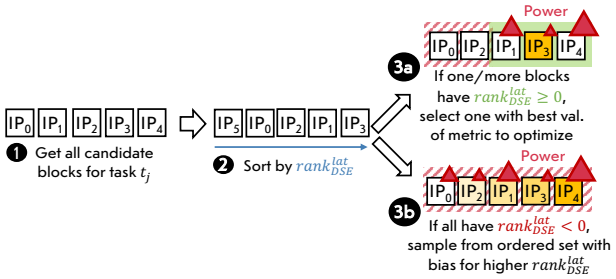

Fig. 5. Real-time constraints aware block selection policy in ARTEMIS. Once a candidate task, t_j, has been selected for optimization in the next DP, we get all the candidate blocks, *i.e.,* the PEs that can execute task t_j, and sort them by $rank^{lat}_{DSE}$.

2) Selection of the Candidate Task: Once a metric is selected, an obvious way to order the tasks to be optimized for the next design point is to weigh this metric for the task *vs.* those of all the other tasks. However, this does not take into account how much the task's acceleration would contribute to its host DAG's slack. Instead, we use the task deadline for task selection and define the $rank^{lat}_{DSE}$ of a task j (not to be confused with scheduling rank) in DAG k as:

$$rank^{lat}_{DSE} = latency_{t_j} \cdot [\underbrace{SDR_{t_j} \cdot T_{dline,DAG_k}}_{\text{sub-deadline of task } t_j} - \underbrace{(T_{wait,t_j} + latency_{t_j})}_{\text{total time spent by task } t_j}] \quad (4)$$

where $latency_{t_j}$ is the latency of the task t_j in the previous design point, $T_{dline,j}$ is the deadline of task t_j, and $T_{wait,j}$ is the amount of time that t_j has been waiting since it became ready. Note that $T_{wait,j}$+$latency_{t_j}$ is the time spent by task t_j in the system. $rank^{lat}_{DSE}$ for a task is $\geq$0 if the task met its deadline, and <0 otherwise. Multiplying with $latency_{t_j}$ ensures that longer-latency tasks are still prioritized for acceleration. Thus, this metric ranks tasks that do not meet their deadline higher than those that do, and, in particular, implicitly ranks tasks with $rank_{het}$ of 4 at the top, as they have high $T_{wait,j}$ through task procrastination. For area, and power, the corresponding ranks of task t_j are the weighted averages across the hardware blocks (PEs, interconnects and memories):

$$rank^{area}_{DSE} = \frac{\sum_{b \in blocks(t_j)} area_{b,t_j}}{\sum_i \sum_{b' \in blocks(t_i)} area_{b',t_i}} \quad (5)$$

$$rank^{pow}_{DSE} = \frac{\max_{b \in blocks(t_j)} pow_{b,t_j}}{\sum_i \max_{b' \in blocks(t_i)} pow_{b',t_i}} \quad (6)$$

Here, $blocks(t_j)$ are the hardware blocks that are active during the execution of task t_j. pow_{b,t_j} and $area_{b,t_j}$ are the power and area, respectively, consumed by block b for task t_j.

3) Selection of PE Blocks: Our explorer selects a PE for optimization if it deems that this PE is the bottleneck to accelerating the task that was selected based on $rank_{DSE}$. The logic for block selection is especially important when the space of design choices is large. In order to extract a good subset of this particular search sub-space, we propose a variation of

Eq. 4, where $latency_{t_j}$ is calculated for each of the candidate blocks, b_i. However, because we may not have observed a prior design point that had b_i, we approximate $latency_{t_j}$ based on the compute latency of t_j on b_i. Our real-time-aware block select logic is illustrated in Fig. 5. The metrics to improve upon are first ranked by importance. Blocks b_i with positive $rank^{lat}_{DSE}$ that will meet the task deadline of t_j are filtered in, and if there are multiple such blocks, we pick the one that improves upon the current block for the most important metric. If no blocks have $rank^{lat}_{DSE}$>0, then we do a pseudo-random Pareto analysis by sampling a candidate block with bias for the highest-ranked block (to increase the likelihood that the design point meets *all* deadlines). This is done in lieu of sampling based on a composite metric (*e.g.,* energy), as a choice that improves the energy of the task in isolation does not necessarily improve the energy of the DAG.

IV. Methodology

This section outlines our input model and methodology for evaluating ARTEMIS.

A. Input Models of Real-Time Applications

As discussed in Sec. II-C, real-time applications are typically modeled as DAGs, where a node (task) is an independent unit of work that can execute when its parents have completed. The DAGs themselves are generated using compiler techniques and/or manual annotation of the application code [15]. Each DAG has a deadline by which it needs to complete, and different DAGs can have their own deadlines.

Typical real-time systems prioritize meeting 100% deadlines while respecting power and area constraints. In the case AVs, for example, it is imperative to meet 100% deadlines to comply with safety regulations [18]. For AR/VR, low/inconsistent frame rates and missed deadlines can lead to motion sickness [2]. Specifically, SoCs need to be designed to guarantee deadlines for critical and periodic DAGs.

We consider several real-world–inspired use cases to evaluate ARTEMIS. These are shown in Figure 6 (left). Each *use case* is composed of one or more *workloads*, where a workload consists of either overlapping or sequential DAGs. The composition of each DAG is shown in Figure 6 (right). We classify these use cases on the basis of arrival pattern and heterogeneity. The application may be *periodic*, where DAGs arrive with fixed inter-arrival time, or *stochastic*, where the DAGs arrive following a probabilistic process (*e.g.,* Poisson [10]). We further classify a use case as *homogeneous* or *heterogeneous*, based on whether it consists of DAGs with identical deadlines and task compositions, or a mix of different DAGs. We, however, do not consider the execution of mixed-criticality DAGs in this work and leave this for a future work.

Based on this taxonomy, we evaluate three use cases spanning different real-time domains. The first is a periodic, homogeneous DAG-based use case called **ERA**. The task composition and DAG structure is derived from [3] using the HPVM compiler [15]. Here, DAG arrivals represent batched sensor inputs from the camera, radar and Wi-Fi receivers of the AV, while the deadline is defined by the maximum time before an action must be taken, *e.g.,* applying the vehicle's brakes. We use this for a detailed evaluation of ARTEMIS on both the *unconstrained* and *fixed-template* modes. We sweep the DAG inter-arrival time, as well as the *vehicle capability* (*i.e.,* number of Viterbi decoding and radar tasks per DAG), for scalability studies.

We also consider an augmented and virtual reality (**AR/VR**) use case composed of workloads from ILLIXR [23] and CAVA [1]. Here, audio decoding plays back the audio based on the user's pose while CAVA's vision pipeline pre-processes images to feed to a neural network backend. Edge detection is used to find sharp changes in brightness in input images and detect objects of interest. This set of workloads are also evaluated in other AR/VR suites, such as [32].

Lastly, we evaluate the audio decoder workload in the context of a different scenario that models a set of **Audio Decoding** DAGs arriving at the SoC in a stochastic fashion. This is representative of a signal processing application that, for instance, mixes audio signals from different sources in real-time. Here, the deadline is a soft deadline and is defined by acceptable lags in audio perception. We re-use the task composition and deadline of the workload from the AR/VR use case, as the exact choice of these is orthogonal to the focus of this work. We model the stochasticity by sampling the DAG inter-arrival time from an exponential distribution [20] with the rate parameter, λ, set to the arrival rate.

B. Input Characterization

We present our methodology for characterization of our hardware library (choice of PEs, memories, *etc.*) and the constraints pertaining to the domains of AVs, and AR/VR.

Hardware Library Characterization. Our methodology allows for importing the hardware library from various sources, such as simulators, chip measurements, analytical models, *etc.*, and the preferred source depends upon the maturity level of the project and availability of characterization data. In Tab. II, we report the details of library sources used in this work. For the ERA workload, we obtained measured power, area and latency data of each of the IP blocks from the 12 nm fabricated chip report in [25]. Note that the chip was characterized using the same ERA workload. We calculate the inter-task data transfer sizes based on the descriptions in [25] and [3]. For the audio decoding, CAVA and edge detection DAGs, we use the open-source latency, power, area and data movement results from [8], which are based on scaled data from AccelSeeker [44]. We note that this data was scaled to a 5 nm process in [8] and we add in a 2-sigma overhead in order to consider the worst-case execution time of the tasks. Thus, we evaluate the SoCs for the two domains (AV and AR/VR) independently, in 12 nm and 5 nm, respectively, to prevent further approximations from technology scaling factors. The IP database for this workload also includes IPs instantiated with different loop-level parallelism (LLP) values, corresponding to different degrees of loop unrolling, which offer greater performance, but also have higher power and

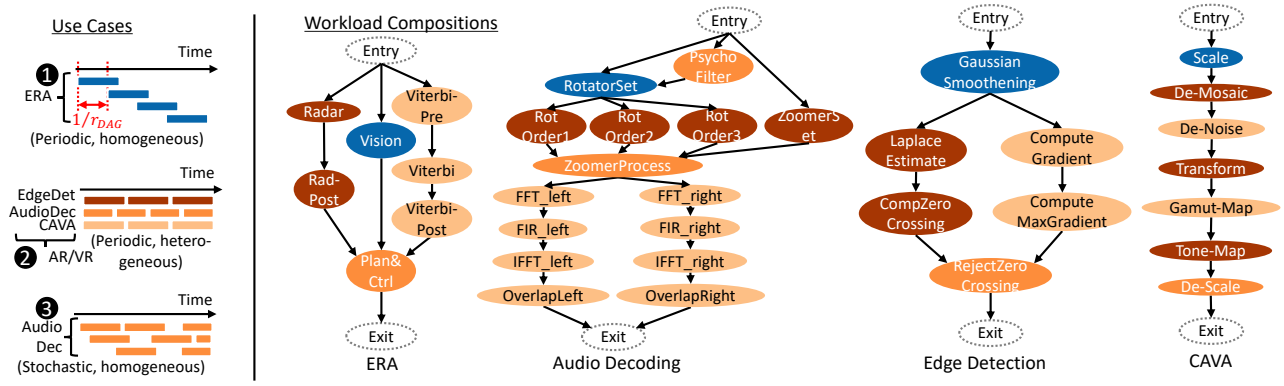

Fig. 6. *Left.* **Use Cases** for evaluation: Autonomous Vehicles (AVs), Augmented/Virtual Reality (AR/VR) and stochastic audio decoding (a generic signal processing domain), with r_{DAG} as the DAG arrival rate. *Right.* Compositions of the unique DAGs that forms each use-case. Edges indicate data dependencies and data movement between tasks. Each task may itself have parallellizable loop iterations, *i.e.,* loop-level parallelism (LLP).

TABLE II
HARDWARE LIBRARY CHARACTERIZATION DATA.

Workload	Base Clock (MHz)	Clock Scaling %	Technology (nm)
ERA	1200	20, 40, ..., 100	12
AR/VR	1000	20, 40, ..., 100	5
Workload	**NoC Width (B)**	**Memory Type**	**Characterization Source**
ERA	4, 8, ..., 256	SRAM	Measured [25]
AR/VR	4, 8, ..., 128	SRAM, DRAM	Simulated

area requirements. We assume independent clock domains for each IP, memory and IC. We do not perform extensive microarchitecture-level DSE in this work, as we believe this DSE can be performed first at a lower level of abstraction and then integrated as an input for the SoC-level DSE.

Constraint Characterization. The scenarios evaluated in this paper and their constraints are in Tab. III. For the ERA use case, we derived the DAG deadline to be 50 ms from [18]. Here, we do not set specific power and area budgets, rather we configure the frameworks to perform best-effort optimization for these metrics *while meeting 100% deadlines*. This is motivated by the safety-criticality constraints of AVs. In the AR/VR use case, we consider a DAG's deadline to be $1/r_{DAG}$ and select r_{DAG} as 50 Hz for Audio Decoding, and 30 Hz for Edge Detection and CAVA, based on ILLIXR [23]. We set the power budget for the SoC to be 200 mW, which is based on the ideal VR headset budget in [23].

C. Simulator and Explorer

ARTEMIS augments an existing SoC simulator [8] with multiple scheduling policies along with our proposed optimizations. It uses a roofline model called Gables [21] to determine if a task is constrained by memory, IC, or PE speed. To ensure fair comparisons against our baselines, we leverage the architecture-aware SA approach [8] for selecting the next design point during exploration in our policy, yet ARTEMIS remains compatible with any other search algorithm. Note that our contributions impact the definition of the neighboring points of a design point, whereas the algorithm for selecting a design point among the neighbors (*e.g.,* SA or RL) is an orthogonal subject. We do not implement hardware coherence support in ARTEMIS and instead assume that the coherence is managed through software, as is typical of scratchpad-based and many-core systems [17], [35], [36], [45].

Exploration Modes. ARTEMIS supports DSE under the *unconstrained* and *fixed-template* modes. In the unconstrained mode, ARTEMIS grows the SoC starting from a seed design point with 1 CPU, IC and memory. In the fixed-template mode, ARTEMIS starts with a seed design comprising CPUs and memories in a fixed NoC topology, and iterates over this design point without changing the topology.

D. SoC Exploration and Deployment Setup

ARTEMIS operates in two functional modes: SoC *exploration*, and simulated *deployment*. We perform DSE using the worst-case number of DAGs that the SoC will observe at any point during deployment. With a DAG deadline of T_{dline} in our homogeneous use cases, we calculate this number as $N_{DAGs,exp} = \lceil T_{dline} \cdot r_{DAG,dep} \rceil$, where $r_{DAG,dep}$ is the DAG arrival rate at deployment. Designing for this worst-case guarantees that the final system can meet all deadlines for the given arrival rate when the DAGs arrive at rate $r_{DAG,dep}$, if the DSE converges. Our deployment runs involve a larger number of DAGs than used for exploration (see Tab. III). We simulate the final design point using our ART_{DSE} policy (Sec. III-B). This policy is implemented within our repository using an API that is similar to that proposed in [40], [41].

TABLE III
SCENARIOS EVALUATED IN THIS PAPER.

DAG Type	Explr. Mode	Use Case	Deadline (ms)	Power Budget	$N_{DAGs,dep}$
Homogeneous, Periodic	Fixed Template	ERA	50	Best effort	50
Homogeneous, Periodic	Unconstrained	ERA	50	Best effort	50
Heterogeneous, Periodic	Unconstrained	AR/VR	21 (AudioDec) 34 (CAVA) 34 (EdgeDet)	200 mW	34 (AudioDec) 21 (CAVA) 21 (EdgeDet)
Homogeneous, Stochastic	Unconstrained	Audio Decoding	21	200 mW	50

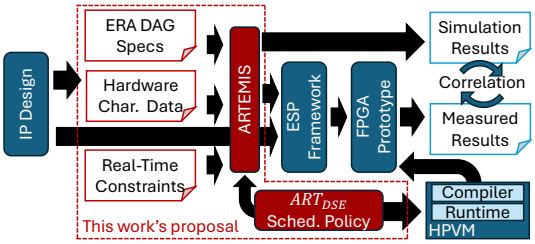

Fig. 7. End-to-end demonstration of ARTEMIS using an FPGA prototype. The output configuration of the design produced by ARTEMIS is fed to the ESP framework to generate an FPGA design that then uses the same scheduling policy, ART_{DSE}, built into the HPVM compiler.

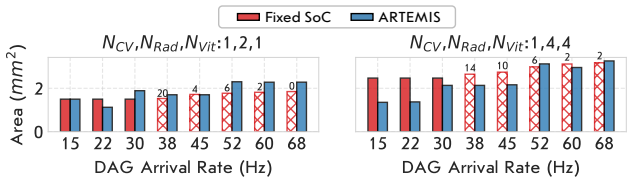

Fig. 8. Area of SoC_{ART} *vs.* SoC_{fixed}. % deadlines met are shown on hatched bars and solid bars show design points that meet all deadlines (critical for AVs). The results are shown for two vehicle capability considerations (*left*: 2 radar sensors and 1 Viterbi decoding engine, *right*: 4 radar sensors and 4 Viterbi decoding engines).

For DSE time, we report the maximum across 10 independent runs of each framework. We compare DSE wall time instead of iteration count, in order to fairly account for the fact that our time-per-iteration is greater than that of prior work, due to the inclusion of dynamic instead of static scheduling in simulations. We define the best design point as the one that meets maximum DAG deadlines with the least power and area, and run simulated deployment on it.

Choice of Baselines. We implement a set of baselines that we categorize into *(i)* techniques that consider mapping as part of the design space and iteratively attempt to converge to an optimal design point, and *(ii)* pseudo-grid-search–like techniques that prune the design space to compute a set of candidate design points and evaluate them.

FARSI [8], **MOOS** [12] and **SA**, belong to category *(i)*. FARSI is a SoTA DSE framework that adapts SoC architecture-aware exploration moves atop standard hill-climbing based SA. MOOS uses a tree model to select a starting Pareto point, and then a local greedy search for expanding the set of starting points. In category *(ii)*, a **two-phase** heterogeneous MPSoC mapping technique [27] is a close prior work; most other works in this space are on static mapping for CGRAs, which is not the target of our work. [27] first performs analytical estimation to compute the number of CPUs and IPs, maps tasks onto them using modulo scheduling, and prunes the generated design space based on static performance estimation.

Finally, we also seek to gauge the benefits obtained with the use of dynamic scheduling *for DSE* as opposed to using it with an *apriori* SoC design during deployment. Therefore, we perform some experiments with FARSI-generated SoC design points that use our ART_{DSE} scheduling policy only during the simulation phase, calling it ARTEMIS/No-DSE.

E. ARTEMIS *End-to-End Use Case and Validation*

This section presents a demonstration of the overall methodology of using ARTEMIS for end-to-end SoC design and evaluation. We illustrate this flow for a specific use-case in Fig. 7. We first obtained hardware implementations of several PEs from [25], including a RISC-V Ariane CPU, and HLS-generated Viterbi Decoding and radar IPs for the FPGA. We created a definition of the ERA workload and associated real-time constraints (Sec. IV-B). Further, for each task in the ERA DAG, we profiled its execution time on eligible PEs. We fed this information into ARTEMIS, exploring three r_{DAG} values (6, 8, and 10 Hz) with the mode set to *fixed-template* of a 2×2 mesh. Next, we synthesized the SoC layout suggested by ARTEMIS using the ESP [34] framework with the original IP implementations, and evaluated it on a Xilinx Virtex VCU118 FPGA running at 78 MHz. To run the ERA workload on the FPGA, we constructed a bare minimal software stack on top of Linux, using the HPVM compiler and runtime [15] and implemented our scheduling policy, ART_{DSE}.

We ran a trace of 50 DAGs on the FPGA with three different periodic arrival rates. We emulated the arrival of the DAGs by invoking 50 threads, each executing an instance of the DAG, and synchronized them by putting the threads to sleep for the required period before waking them to run the ERA workload. In the background, our scheduler built into the HPVM runtime performed scheduling across all of the tasks seen at any given point in time. The FPGA performance was within 10% of the latency estimated by ARTEMIS for the same configuration. We attribute this deviation to a delay in thread wake-up after the `sleep` syscall, which introduced minor lags in task execution. A real-world scenario would use sensors that would directly stream data to the SoC, thereby eliminating this artifical overhead.

Dynamic SoC Scheduler. The dynamic SoC scheduler in the final SoC may be implemented in either hardware or software; for our experiments with the FPGA, we implemented the ART_{DSE} policy within the runtime of the HPVM compiler stack [15]. *Our paper demonstrates that the same scheduler with minor changes in the policy (e.g., using task procrastination instead of dropping the task) can be used to accelerate the DSE.* This "simulated" scheduler that is used during DSE and simulated deployment is entirely a software implementation and we will offer it as part of the release.

V. Evaluation

We compare ARTEMIS with the baselines and evaluate the quality of SoC (power, area and % of deadlines met), and the speed of DSE (wall clock time). In the rest of the paper, we use the notation of $SoC_{framework}$ to denote the final design point produced by a framework. We evaluate the quality of SoCs based on their power consumption, area and sustained DAG throughput. We define sustained DAG throughput, $r_{DAG,sus}$, as the maximum DAG arrival rate for which the SoC meets 100% of the application deadlines.

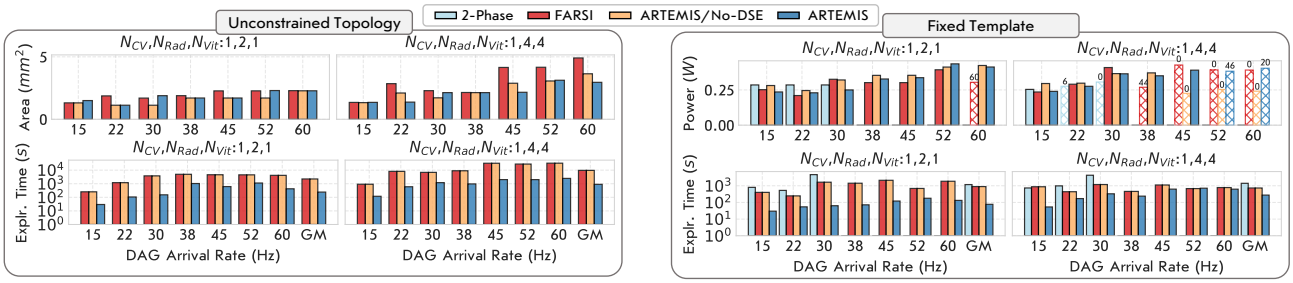

Fig. 9. Area/power and DSE time (log scale) comparison with prior works for two different AV capabilities. % deadlines met is shown on each hatched bar and solid bars show SoCs that meet all deadlines (critical for AVs) *Top.* Unconstrained topology mode. *Bottom.* Fixed-template in 3×3 mesh: 1 I/O slot, 2 memory slots, 6 PE slots.

A. SoC Quality and DSE Time Comparisons

1) ERA Use Case: We consider the ERA use case with varying number of vision (N_{CV}), radar (N_{rad}) and Viterbi (N_{vit}) tasks. Here, we run the exploration for each DAG arrival rate value to generate independent SoCs, and then simulate them on a periodic DAG trace with that arrival rate to emulate a deployment scenario.

Against Fixed SoC. In Fig. 8, we compare the ARTEMIS-generated SoC (SoC_{ART}) with a fixed heterogeneous SoC (SoC_{fixed})[4] that has 1 CPU, N_{CV} CV IPs, N_{rad} radar IPs and N_{vit} Viterbi IPs. This baseline may be considered an "obvious" choice without the use of any DSE tools. At arrival rates below 30 Hz, all of the SoCs meet 100% deadlines. However, SoC_{ART} is generally more area-efficient (up to 2×) compared to SoC_{fixed}. This is because the use cases involving slow arrival rates have large windows of idleness, *e.g.,* 17 ms between DAGs @15 Hz, for which SoC_{fixed} is over-provisioned compared to SoC_{ART}. For faster DAG arrivals, on the contrary, SoC_{fixed} is too under-provisioned to handle multiple overlapping DAGs and thus misses increasingly more deadlines, whereas SoC_{ART} has sufficient IPs to meet all deadlines. At iso-area, SoC_{ART} has a peak sustained throughput, r_{sus}, of 45 Hz, a **1.5**× improvement over SoC_{fixed} (r_{sus}=30 Hz).

Against Baseline-Generated SoCs. We present a comparison of SoC_{ART} with SoCs generated by Jia *et al.* [27] (SoC_{2_phase}) and FARSI [8] (SoC_{FARSI}). Additionally, we compare against a SoC generated by FARSI but deployed with ART_{DSE} ($SoC_{FARSI,dyn}$) to underscore our innovation of utilizing the scheduler for determining the SoC itself. The results for the unconstrained topology mode in Fig. 9 (top) show that SoC_{ART} achieves up to **2.4**× lower area over SoC_{FARSI} and $SoC_{FARSI,dyn}$ (SoC_{FARSI} with ART_{DSE} scheduling). The benefits are more pronounced with higher DAG arrival rates and more tasks per DAG. In terms of DSE time, ARTEMIS achieves speedups of **9.6–13.2**× over FARSI. Note that 2_phase [27] does not work for unconstrained topologies.

The results for the fixed-template mode are shown in Fig. 9 (bottom). Here, note that 2_phase has no solution when the inter-arrival time ($1/r_{DAG}$) is less than the time to execute the slowest task on the fastest PE [27] (for r_{DAG}>30 Hz here). In addition, 2_phase does not scale with arrival rates or number of tasks per DAG, because it does not prune the design space based on real-time metrics. SoC_{ART} achieves similar or lower power than the baselines while achieving higher r_{sus}. In Fig. 9 (bottom-right), for r_{DAG} of 45 Hz, SoC_{ART} has 2 CV, 1 Rad, and 2 Vit IPs, whereas SoC_{FARSI} places 5 Vit IPs thereby producing a design point that misses deadlines. Overall, SoC_{ART} achieves $r_{DAG,sus}$ gains of up to **1.2**×, **1.5**× and **3**× over $SoC_{FARSI,dyn}$, SoC_{FARSI} and SoC_{2_phase}, *resp.*, with **8.4–12.8**× faster DSE speed.

[4]Note that the SoC area increases with r_{DAG} for SoC_{fixed}, because the memory size increases with the number of concurrently executing tasks.

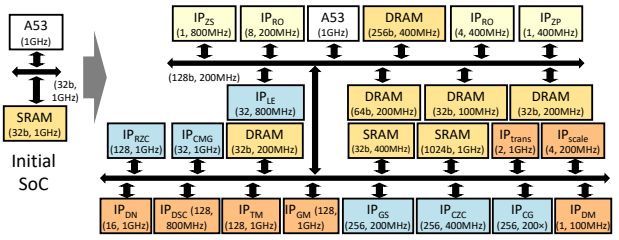

Fig. 10. Block diagrams of initial (left) and final (right) SoC designs generated by ARTEMIS for AR/VR. Numbers in parenthesis (**A**, **B**) indicate LLP (for IPs) or bitwidth (for interconnect/memory) of **A**, and clock frequency of **B**. This SoC was realized without any limit on the number of SRAM/DRAM instances. However, ARTEMIS exposes a knob to the designer to optionally limit the number of instances of any particular block type.

2) AR/VR Use Case: We now evaluate ARTEMIS along with SA, MOOS and FARSI, on the AR/VR use case. Note that 2_phase does not support use cases involving heterogeneous DAGs. We do not evaluate a fixed SoC (as done for ERA), because the choice of what LLP to pick for each task's IP is not obvious.

The final SoC_{ART} for AR/VR (Fig. 10) meets 100% deadlines within the power budget without any IPs for several tasks, such as FFT, FIR, *etc.* Such insights for complex real-time applications may be highly non-obvious.

We report detailed comparison of the best designs generated by each framework in Figure 11 (top), with iso-DSE–time results in Figure 11 (bottom). ARTEMIS finishes exploration **5.1–36.8**× times faster than the baselines. Further, SoC_{ART} has **1.8**× less power and **4.9**× less area than SoC_{FARSI}. This

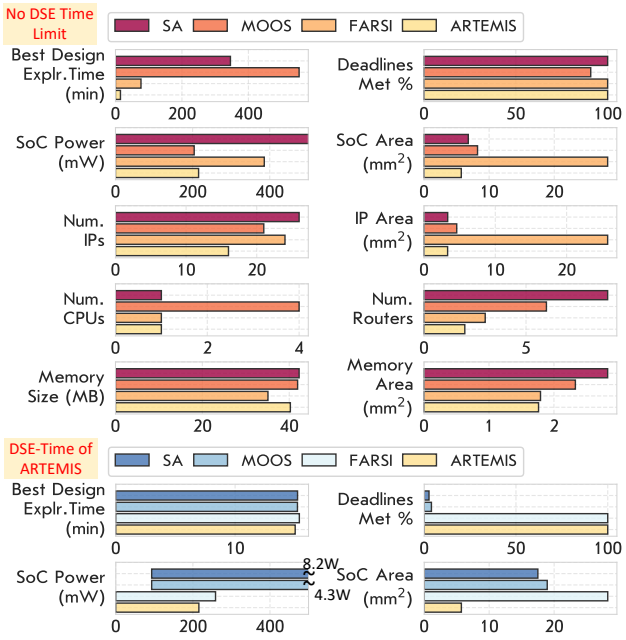

Fig. 11. Comparison with prior works for AR/VR with 1 DAG each for audio decoding, CAVA and edge detection for DSE, and 34, 21 and 21 of the same DAGs, for deployment. The bottom graphs are for the case of running the baselines for the same duration as taken by ARTEMIS for DSE.

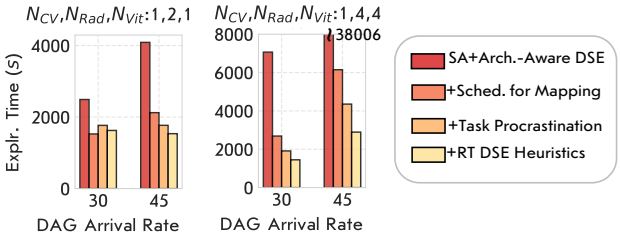

Fig. 13. DSE speed benefits of each feature of ARTEMIS introduced in this work for ERA with varying DAG arrival rates and vehicle capabilities.

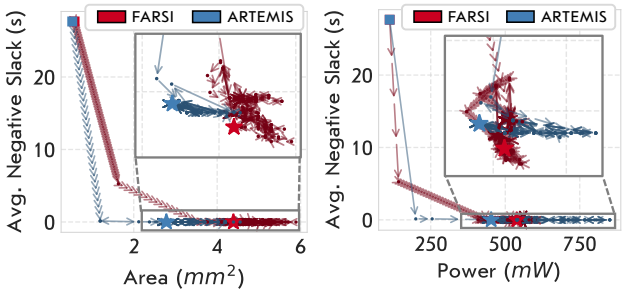

Fig. 14. DSE trail of SoC_{FARSI} and SoC_{ART} for ERA using best-effort estimation. The plots are for AV capability of $(N_{CV}, N_{rad}, N_{vit})$ of (1, 4, 4) and DAG arrival rate of 45 Hz. Squares and stars indicate the first explored SoC and the final SoC, respectively. The number of DSE iterations for ARTEMIS and FARSI, respectively, are 465 and 1,938.

is owed to our real-time constraints aware block selection technique (Section III-C3) that avoids extremely large LLP IPs while still prioritizing performance. The remainder of our gains stem from smaller memory footprint and a compact network topology. MOOS also generates a design with similar area and power, but it misses **9.2%** of the deadlines. Furthermore, the DSE for MOOS takes **36.8**× longer to arrive at this design. If the DSE time is restricted to ARTEMIS's convergence time, FARSI over-provisions, whereas MOOS/SA produce bad design points that neither meet deadlines nor the power budget. This substantiates our claim that ARTEMIS not only performs fast DSE, but also discovers efficient SoCs, by delegating the mapping to the scheduler,

3) Stochastic Audio Decoding Use Case: We now show a use case where ARTEMIS is used to design an SoC to sustain a worst-case DAG arrival rate, but is deployed for stochastic DAGs, *i.e.,* with variable and unbounded r_{DAG}.

Fig. 12 shows our results for the case where we deploy *a single* SoC design that was explored for a 250 Hz arrival rate, on a set of stochastic traces. We observe that the % deadlines met is close to, but not exactly 100% at 250 Hz, although the SoC was designed for this value, which is contrary to the case of periodic DAG arrivals. However, this is expected, because given a long enough trace there always exist "rogue" DAGs that arrive within quick succession of the previous DAG, which can cascade and cause the SoC to miss the next few DAGs' deadlines, until the congestion clears. This can be mitigated using methods such as criticality-aware scheduling, but this is orthogonal to our work.

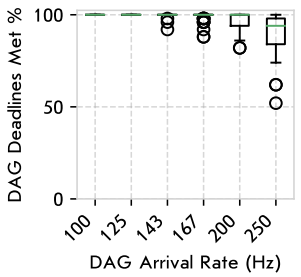

Fig. 12. Distribution of DAG deadlines met across 50 traces and different arrival rates, for Audio Decoding. Traces are simulated on a design point explored using ART_{DSE} with r_{DAG} = 250 Hz.

4) Benefit Breakdown Analysis: In Fig. 13, we report the incremental benefits of each feature introduced in this work.

The baseline policy uses SA with SoC architecture-aware exploration moves for DSE. Our insight of using the scheduler to eliminate the mapping dimension from the DSE offers significant speedup of up to **6.2**×. Task procrastination is beneficial when there is congestion in the workload, and provides an additional **1.4**× DSE speedup for the most congested case ((1,4,4) @45 Hz). Finally, our real-time constraints aware metric, task and block selection heuristics provides an added benefit of up to **1.5**×. The combined results demonstrate a DSE speedup of **1.5–13.1**× over the baseline.

5) DSE Exploration Path Analysis: In Fig. 14, we show a visualization of the iterative DSE process by plotting the convergence trail of the SoC design points explored using FARSI and ARTEMIS, for comparison. We select a representative experiment from the ERA use case with best effort optimization for power and area while meeting all deadlines. The SoC area and peak power are plotted on the x-axis of the two plots, *resp.*. The y-axis shows the design's average nega-

TABLE IV
IMPACT OF SCHEDULING POLICY ON DSE TIME AND SOC POWER, COMPARING ART_{DSE} [THIS WORK] WITH BASELINE POLICIES. NUMBERS IN RED DENOTE NO DEADLINES WERE MET.

		EDF	EFT	MS_{stat}	MS_{dyn}	$\mathbf{ART_{DSE}}$
r_{DAG}=30 Hz	Exploration Time (s)	403	89	108	123	**87**
	Power (mW)	188	402	456	345	**357**
r_{DAG}=90 Hz	Exploration Time (s)	4,496	9,284	13,304	11,494	**8,279**
	Power (mW)	455	844	870	782	**769**

tive slack, *i.e.,* $\frac{1}{N}\sum_{k=0}^{N-1}(-slack_{DAG_k})$, where $slack_{DAG_k}$ is the slack of DAG k (defined in Section III-A), for N DAGs. We observe that ARTEMIS avoids over-provisioning from the start and converges at a much more compact design than FARSI. *E.g.,* FARSI chooses 8 Vit IPs *vs.* 2 Vit IPs, for ARTEMIS. ARTEMIS follows a much steadier trail over FARSI once the average negative slack goes < 0; FARSI spends several iterations backtracking to optimize for power and area. Here, ARTEMIS finishes in 4.6× fewer iterations.

6) Impact of Scheduling Policy on DSE: We evaluate the effect of scheduling policies on SoC exploration using ARTEMIS, comparing ART_{DSE} to prior policies, namely EDF [22], EFT [7], and two HetSched policies, namely, MS_{stat} and MS_{dyn} [4], [5]. In Table IV, we show the exploration time and power results for ERA for a representative AV capability with N_{CV}, N_{rad}, N_{vit} of 1,4,4 and two arrival rates. EDF does not converge to a design meeting 100% deadlines across any of these arrival rates, as it naïvely executes tasks on the fastest available PE without considering other tasks that may be mapped on it, or the possibility of executing the task on a slower PE while meeting its deadline. In contrast, all deadlines are met for the HetSched policies.

We observe that the SoC with the smallest power is the one generated using MS_{dyn} and ART_{DSE}. Between the two, however, DSE with ART_{DSE} is **42%-74%** faster. The differences are starker for the slower-arrival–case, because slow-arriving DAGs allow for more slack for a given task, which we exploit to perform energy optimizations ($rank_{het}$ of 2 and 3 – see Figure 2), that enable the task to run on a lower-energy PE, while still meeting its deadline.

VI. DISCUSSION

We present here some discussion points on where ARTEMIS fits in the overall SoC design flow, the types of guarantees it provides, and the conditions under which such guarantees hold.

ARTEMIS is built as an early-stage DSE utility to augment an SoC architect's workbench. Fundamentally, there exists a trade-off between generality across multiple domains and workloads, and fidelity for a given domain. Because of its generality, ARTEMIS relies on external characterization data (*e.g.,* power and performance of IPs) and input parameters (*e.g.,* DAG arrival rate) in order to generate meaningful results and insights. One can build in some conservativeness into these inputs (such as our latency overhead consideration in Sec. IV-B) in order to ensure stronger guarantees from ARTEMIS. Further, the exploration phase needs to be done with at least the same number of DAGs as would be expected in a real-world deployment scenario; see the $N_{DAGs,exp}$ calculation in Sec. IV-D. Under such considerations, ARTEMIS guarantees that if an SoC design point converged with a sustained DAG throughput of $r_{DAG,sus}$, then it would meet 100% of deadlines when it is deployed with any arbitrary number of DAGs with the same or lower DAG throughput.

VII. CONCLUSION

In this paper, we proposed ARTEMIS, a framework for agile early-stage design space exploration (DSE) of heterogeneous, domain-specific SoCs in the context of real-time applications. The goal of this work is to reduce the DSE convergence time, while improving, if not maintaining, the quality of the generated SoC design in terms of power-performance area metrics.

ARTEMIS employs an real-time, heterogeneity-aware, dynamic scheduler to eliminate the DSE dimension of task-to-PE mapping, thereby reducing the design space size by several orders-of-magnitude. We propose a technique called task procrastination and combine it with energy-aware and memory-traffic aware optimizations to expose bottleneck tasks to the exploration framework, thereby accelerating the search for an optimal SoC design. Additionally, real-time deadline and power/area-aware ranking mechanisms for metric, task and block optimization aid the DSE process directly.

We evaluated ARTEMIS for the autonomous vehicle and augmented/virtual reality domains. Our practical benefits emerge in the **5.1–12.8**× reduction in DSE time, which enables agile SoC discovery within minutes instead of hours. At the same time, ARTEMIS generates SoCs with up to **2.4**× lower area at iso-throughput and up to **3**× better throughput at iso-area over prior work. We believe that the techniques introduced in this paper are a key enabler for fast and efficient DSE of future real-time systems.

ACKNOWLEDGMENT

We thank the anonymous reviewers and artifact evaluators for their valuable feedback, meticulous reviews, and comments, which have allowed us to improve this work considerably. This research was developed with funding from the Defense Advanced Research Projects Agency (DARPA). The views, opinions and/or other findings expressed are those of the authors and should not be interpreted as representing the official views or policies of the Department of Defense or the U.S. Government.

REFERENCES

[1] "CAVA: Camera vision pipeline on gem5-Aladdin," 2019, accessed: 2022-11-18. [Online]. Available: https://github.com/yaoyuannnn/cava

[2] "Ieee standard for head-mounted display (hmd)-based virtual reality(vr) sickness reduction technology," *IEEE Std 3079-2020*, pp. 1–74, 2021.

[3] "Mini-ERA: Simplified version of the main ERA workload," 2021, accessed: 2022-07-04. [Online]. Available: https://github.com/IBM/mini-era

[4] A. Amarnath, S. Pal, H. Kassa, A. Vega, A. Buyuktosunoglu, H. Franke, J.-D. Wellman, R. Dreslinski, and P. Bose, "Heterogeneity-aware scheduling on SoCs for autonomous vehicles," *IEEE Computer Architecture Letters*, vol. 20, no. 2, pp. 82–85, 2021.

[5] A. Amarnath, S. Pal, H. Kassa, A. Vega, A. Buyuktosunoglu, H. Franke, J.-D. Wellman, R. Dreslinski, and P. Bose, "HetSched: Quality-of-mission aware scheduling for autonomous vehicle SoCs," *arXiv preprint arXiv:2203.13396*, 2022.

[6] T. K. Bandara, D. Wijerathne, T. Mitra, and L.-S. Peh, "Revamp: A systematic framework for heterogeneous cgra realization," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 918–932. [Online]. Available: https://doi.org/10.1145/3503222.3507772

[7] J. Blythe, S. Jain, E. Deelman, Y. Gil, K. Vahi, A. Mandal, and K. Kennedy, "Task scheduling strategies for workflow-based applications in grids," in *CCGrid 2005. IEEE International Symposium on Cluster Computing and the Grid, 2005.*, vol. 2. IEEE, 2005, pp. 759–767.

[8] B. Boroujerdian, Y. Jing, D. Tripathy, A. Kumar, L. Subramanian, L. Yen, V. Lee, V. Venkatesan, A. Jindal, R. Shearer *et al.*, "FARSI: An early-stage design space exploration framework to tame the domain-specific system-on-chip complexity," *ACM Transactions on Embedded Computing Systems (TECS)*, 2022.

[9] M. Cochet, K. Swaminathan, E. Loscalzo, J. Zuckerman, M. C. Dos Santos, D. Giri, A. Buyuktosunoglu, T. Jia, D. Brooks, G.-Y. Wei *et al.*, "Blitzcoin: Fully decentralized hardware power management for accelerator-rich socs," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 801–817.

[10] P. C. Consul and G. C. Jain, "A generalization of the poisson distribution," *Technometrics*, vol. 15, no. 4, pp. 791–799, 1973.

[11] D. Derafshi, A. Norollah, M. Khosroanjam, and H. Beitollahi, "Hrhs: A high-performance real-time hardware scheduler," *IEEE Transactions on Parallel and distributed Systems*, vol. 31, no. 4, pp. 897–908, 2019.

[12] A. Deshwal, N. Jayakodi, B. Joardar, J. Doppa, and P. Pande, "MOOS: A multi-objective design space exploration and optimization framework for NoC enabled manycore systems," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 18, no. 5s, pp. 1–23, 2019.

[13] R. Dick and N. Jha, "MOGAC: a multiobjective genetic algorithm for hardware-software cosynthesis of distributed embedded systems," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 17, no. 10, pp. 920–935, 1998.

[14] M. C. Dos Santos, T. Jia, J. Zuckerman, M. Cochet, D. Giri, E. J. Loscalzo, K. Swaminathan, T. Tambe, J. J. Zhang, A. Buyuktosunoglu *et al.*, "14.5 a 12nm linux-smp-capable risc-v soc with 14 accelerator types, distributed hardware power management and flexible noc-based data orchestration," in *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 67. IEEE, 2024, pp. 262–264.

[15] A. Ejjeh, A. Councilman, A. Kothari, M. Kotsifakou, L. Medvinsky, A. Noor, H. Sharif, Y. Zhao, S. Adve, S. Misailovic, and V. Adve, "HPVM: Hardware-agnostic programming for heterogeneous parallel systems," *IEEE Micro*, vol. 42, no. 5, pp. 108–117, 2022.

[16] P. Eles, Z. Peng, K. Kuchcinski, and A. Doboli, "System level hardware/software partitioning based on simulated annealing and tabu search," *Design Automation for Embedded Systems*, vol. 2, no. 1, pp. 5–32, 1997.

[17] S. Feng, J. Sun, S. Pal, X. He, K. Kaszyk, D.-h. Park, M. Morton, T. Mudge, M. Cole, M. O'Boyle *et al.*, "Cosparse: A software and hardware reconfigurable spmv framework for graph analytics," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 949–954.

[18] I. Gog, S. Kalra, P. Schafhalter, J. E. Gonzalez, and I. Stoica, "D3: a dynamic deadline-driven approach for building autonomous vehicles," in *Proceedings of the Seventeenth European Conference on Computer Systems*, 2022, pp. 453–471.

[19] A. A. Goksoy, A. Krishnakumar, M. S. Hassan, A. J. Farcas, A. Akoglu, R. Marculescu, and U. Y. Ogras, "Das: Dynamic adaptive scheduling for energy-efficient heterogeneous socs," *IEEE Embedded Systems Letters*, vol. 14, no. 1, pp. 51–54, 2021.

[20] R. D. Gupta and D. Kundu, "Generalized exponential distribution: different method of estimations," *Journal of Statistical Computation and Simulation*, vol. 69, no. 4, pp. 315–337, 2001.

[21] M. Hill and V. Reddi, "Gables: A roofline model for mobile SoCs," in *Proceedings of the 2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2019, pp. 317–330.

[22] J. Hong, X. Tan, and D. Towsley, "A performance analysis of minimum laxity and earliest deadline scheduling in a real-time system," *IEEE Transactions on Computers*, vol. 38, no. 12, pp. 1736–1744, 1989.

[23] M. Huzaifa, R. Desai, S. Grayson, X. Jiang, Y. Jing, J. Lee, F. Lu, Y. Pang, J. Ravichandran, F. Sinclair, B. Tian, H. Yuan, J. Zhang, and S. Adve, "ILLIXR: Enabling end-to-end extended reality research," in *Proceedings of the 2021 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2021, pp. 24–38.

[24] International Organization for Standardization (ISO), "Road vehicles — functional safety." https://www.iso.org/standard/68383.html.

[25] T. Jia, P. Mantovani, M. Dos Santos, D. Giri, J. Zuckerman, E. Loscalzo, M. Cochet, K. Swaminathan, G. Tombesi, J. Zhang, N. Chandramoorthy, J.-D. Wellman, K. Tien, L. Carloni, K. Shepard, D. Brooks, G.-Y. Wei, and P. Bose, "A 12nm agile-designed SoC for swarm-based perception with heterogeneous IP blocks, a reconfigurable memory hierarchy, and an 800MHz multi-plane NoC," in *Proceedings of the 48th European Solid State Circuits Conference (ESSCIRC)*. IEEE, 2022, pp. 269–272.

[26] Z. Jia, A. Pimentel, M. Thompson, T. Bautista, and A. Núñez, "NASA: A generic infrastructure for system-level MP-SoC design space exploration," in *Proceedings of the 8th IEEE Workshop on Embedded Systems for Real-Time Multimedia*, 2010, pp. 41–50.

[27] Z. J. Jia, A. Núñez, T. Bautista, and A. D. Pimentel, "A two-phase design space exploration strategy for system-level real-time application mapping onto mpsoc," *Microprocessors and Microsystems*, vol. 38, no. 1, pp. 9–21, 2014.

[28] J. Keller, S. Litzinger, and C. Kessler, "Combining design space exploration with task scheduling of moldable streaming tasks on reconfigurable platforms," in *Proceedings of the 17th International Symposium on Applied Reconfigurable Computing*. Springer, 2021, pp. 93–107.

[29] M. Kim, S. Banerjee, N. Dutt, and N. Venkatasubramanian, "Design space exploration of real-time multi-media MPSoCs with heterogeneous scheduling policies," in *Proceedings of the 4th International Conference on Hardware/Software Codesign and System Synthesis*, 2006, p. 16–21.

[30] S. Krishnan, A. Yazdanbakhsh, S. Prakash, J. Jabbour, I. Uchendu, S. Ghosh, B. Boroujerdian, D. Richins, D. Tripathy, A. Faust *et al.*, "Archgym: An open-source gymnasium for machine learning assisted architecture design," in *ISCA*. IEEE, 2023.

[31] P. Kuacharoen, M. Shalan, and V. J. Mooney, "A configurable hardware scheduler for real-time systems." in *Engineering of Reconfigurable Systems and Algorithms*, 2003, pp. 95–101.

[32] H. Kwon, K. Nair, J. Seo, J. Yik, D. Mohapatra, D. Zhan, J. Song, P. Capak, P. Zhang, P. Vajda, C. Banbury, M. Mazumder, L. Lai, A. Sirasao, T. Krishna, H. Khaitan, V. Chandra, and V. J. Reddi, "Xrbench: An extended reality (xr) machine learning benchmark suite for the metaverse," 2022. [Online]. Available: https://arxiv.org/abs/2211.08675

[33] M. Lukasiewycz, M. Streubuhr, M. Glaß, C. Haubelt, and J. Teich, "Combined system synthesis and communication architecture exploration for MPSoCs," in *Proceedings of the 2009 Design, Automation & Test in Europe Conference & Exhibition*. IEEE, 2009, pp. 472–477.

[34] P. Mantovani, D. Giri, G. Di Guglielmo, L. Piccolboni, J. Zuckerman, E. G. Cota, M. Petracca, C. Pilato, and L. P. Carloni, "Agile soc development with open esp," in *Proceedings of the 39th International Conference on Computer-Aided Design*, 2020, pp. 1–9.

[35] S. Pal, A. Amarnath, S. Feng, M. O'Boyle, R. Dreslinski, and C. Dubach, "Sparseadapt: Runtime control for sparse linear algebra on a reconfigurable accelerator," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 1005–1021.

[36] S. Pal, S. Feng, D.-h. Park, S. Kim, A. Amarnath, C.-S. Yang, X. He, J. Beaumont, K. May, Y. Xiong *et al.*, "Transmuter: Bridging the efficiency gap using memory and dataflow reconfiguration," in *Proceedings*

of the ACM International Conference on Parallel Architectures and Compilation Techniques, 2020, pp. 175–190.
[37] G. Palermo, C. Silvano, and V. Zaccaria, "Discrete particle swarm optimization for multi-objective design space exploration," in *Proceedings of the 11th EUROMICRO Conference on Digital System Design Architectures, Methods and Tools*. IEEE, 2008, pp. 641–644.
[38] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
[39] C. Tan, T. Tambe, J. J. Zhang, B. Fang, T. Geng, G.-Y. Wei, D. Brooks, A. Tumeo, G. Gopalakrishnan, and A. Li, "Asap: Automatic synthesis of area-efficient and precision-aware cgras," in *Proceedings of the 36th ACM International Conference on Supercomputing*, ser. ICS '22. New York, NY, USA: Association for Computing Machinery, 2022. [Online]. Available: https://doi.org/10.1145/3524059.3532359
[40] A. Vega, A. Amarnath, J.-D. Wellman, H. Kassa, S. Pal, H. Franke, A. Buyuktosunoglu, R. Dreslinski, and P. Bose, "Stomp: A tool for evaluation of scheduling policies in heterogeneous multi-processors," *arXiv preprint arXiv:2007.14371*, 2020.
[41] A. Vega, J.-D. Wellman, H. Franke, A. Buyuktosunoglu, P. Bose, A. Amarnath, H. Kassa, S. Pal, and R. Dreslinski, "Stomp: Agile evaluation of scheduling policies in heterogeneous multi-processors," in *DOSSA-3 Workshop@ HPCA*, 2021.
[42] P. Wu and M. Ryu, "Best speed fit EDF scheduling for performance asymmetric multiprocessors," *Mathematical Problems in Engineering*, vol. 2017, 2017.
[43] X.-J. Xu, C.-B. Xiao, G.-Z. Tian, and T. Sun, "Hybrid scheduling deadline-constrained multi-DAGs based on reverse HEFT," in *Proceedings of the 2016 International Conference on Information System and Artificial Intelligence (ISAI)*. IEEE, 2016, pp. 196–202.
[44] G. Zacharopoulos, L. Ferretti, G. Ansaloni, G. Di Guglielmo, L. Carloni, and L. Pozzi, "Compiler-assisted selection of hardware acceleration candidates from application source code," in *Proceedings of the 37th International Conference on Computer Design (ICCD)*, 2019, pp. 129–137.
[45] X. Zhou, H. Chen, S. Luo, Y. Gao, S. Yan, W. Liu, B. Lewis, and B. Saha, "A case for software managed coherence in manycore processors," in *Poster on 2nd USENIX Workshop on Hot Topics in Parallelism HotPar10*, 2010.

APPENDIX

A. Abstract

We provide an artifact to reproduce the results for Fixed SoC, ARTEMIS, FARSI, and ARTEMIS/No-DSE in Fig. 8, Fig. 9, and Table IV. Due to the large running time of the experiments and being considerate of the evaluators' time, we limit the suggested experiments to these three sets which we consider to be the key results of this paper. The source code for ARTEMIS is open-source and available at https://github.com/IBM/ARTEMIS-DSE.

B. Artifact check-list (meta-information)

- **Program:** Python 3.x and conda
- **Run-time environment:** Tested on RHEL8, but should work with any standard Linux distro
- **Hardware:** We used a Xeon Gold 6258R CPU with 64 GB of memory, but any multi-threaded CPU with at least 8–16 threads and a similar amount of memory should work
- **Metrics:** Exploration time for the framework and simulated SoC power, area, deadlines met %
- **Output:** CSV files, PDF figures and a table
- **Experiments:** Experiments done to reproduce Fig. 8, Fig. 9, and Table IV
- **How much disk space required (approximately)?:** $\sim$2 GB
- **How much time is needed to prepare workflow (approximately)?:** Cloning the repository and bootstrapping the setup process should take less than 30 mins.
- **How much time is needed to complete experiments (approximately)?:** $\sim$2 days ($\sim$4 days with optional experiments)
- **Publicly available?:** Yes
- **Code licenses (if publicly available)?:** MIT License
- **Archived (provide DOI)?:** Will provide once accepted.

C. Description

1) How to access: The code is publicly available at https://github.com/IBM/ARTEMIS-DSE.

2) Hardware dependencies: The artifact may be evaluated on any multi-threaded machine with a decent amount of main memory (64 GB suggested). Please make note of the expected variability in the results that we mention in Sec. F below.

3) Software dependencies: The code can be run on any Linux system. We have tested it on a RHEL8 machine. The instructions in the README create a conda environment called `artemis` that contains all of the necessary Python packages. No other dependencies exist.

D. Installation

To set up the repository, please follow the instructions in the **Quick Setup** section of the README in https://github.com/IBM/ARTEMIS-DSE. Running `bootstrap.sh` as described in the README takes care of the setup. Please ensure that at the end of this step you are within the conda environment named `artemis`.

E. Experiment workflow

Once the steps in the **Quick Setup** section are complete, please go into `generators/scripts` and open the file `ae_run_launch.py`. This is the top-level run script. We have segregated the experiments into two sets: Set 1 covers the experiments for Fig. 8 and Fig. 9, while Set 2 covers the experiments for Table IV.

There are lower-level scripts that collect the results of the framework's execution into CSV files. For convenience, we have provided higher-level scripts to plot the results and allow the evaluators to compare visually against the figures/table in this paper.

F. Evaluation and expected results

- From `generators/scripts/`, invoke the `python3 ae_run_launch.py` script and the experiments should start running. This will take $\sim$2 days to finish. Therefore, please make sure that the process continues running using utilities such as `tmux` or `screen`. Please note that this will run only the even-numbered data points (DAG arrival rates) of Fig. 8 and Fig. 9. Optionally, you may set `reduced_exp_set` to `False` to reproduce the experiments for all of the data points in the figures, but please note that it will take $\sim 2\times$ the time to complete.
- Once the runs have finished, run `python3 ae_plot_comp.py` and then `sh ae_collect.sh`. This will generate the PDFs corresponding to Fig. 8 and Fig. 9 in `paper_results/`. Please note that you may run this step while the experiments are in progress to observe intermediate results, if desired. Also, ensure that the setting for `reduced_exp_set` mirrors that of the previous step.

- Finally, run `python3 ae_tabulate_sched.py` which will display the results corresponding to Table IV.

Notes. In this artifact, we focus on two metrics-of-interest: **(1)** power/area/deadlines met % of the generated SoC, and **(2)** exploration time, which is the wall time of the framework on the host machine. The values for **(1)** are expected to be close, but not match precisely, with the results reported in this paper. This is because of effects of multi-threading and randomness of the design space exploration algorithm in the code-base. However, the trends and takeaways are expected to remain the same. The results for **(2)** are expected to have even greater difference if the configuration of the host system differs significantly from what we used, and/or if the measurements were taken in isolation. *E.g.*, one of our AE evaluators reported up to $10\times$ difference in absolute wall clock times, compared to the results reported in our paper. *Therefore, we urge the readers to consider the relative results and the trends, rather than the absolute values.*

G. Experiment customization

While we have set the default values in our scripts to replicate the experiments in this paper, we allow for customization by changing the knobs in file names mentioned below:

- [`ae_run_launch.py`] Change the *x* and *y* dimensions of the mesh structure by changing the first two parameters of each tuple in `soc_x_y_dag_iat_all`. Please note that these parameters are ignored for the unconstrained topology mode (`CONSTRAIN_TOPOLOGY` of 0). Also, ensure to change the same variable in the plot script `ae_plot_comp.py`.
- [`ae_run_launch.py`] Change the inter-arrival time of the DAGs in the use-case by changing the third parameter of each tuple in `soc_x_y_dag_iat_all`. Also, ensure to change the same variable in the plot script `ae_plot_comp.py`.
- [`ae_run_launch.py`] Change the vehicle capability of the simulated AV in the ERA workload, by changing the values for N_{CV}, N_{rad} and N_{vit} in the variable `ncv_nrad_nvit`. Also, ensure to change the same variable in the plot script `ae_plot_comp.py`.
- [`gen_parse_data.py`] Change the per-DAG deadline, and the power and area budgets, by redefining the variable `def_budgets`.
- [`gen_parse_data.py`] Change the per-PE latency, power and area, by redefining the variable `ppa_dict`.